

SIT314

Project Report

A Scalable, AI-Powered Anomaly Detection System For Bank Security

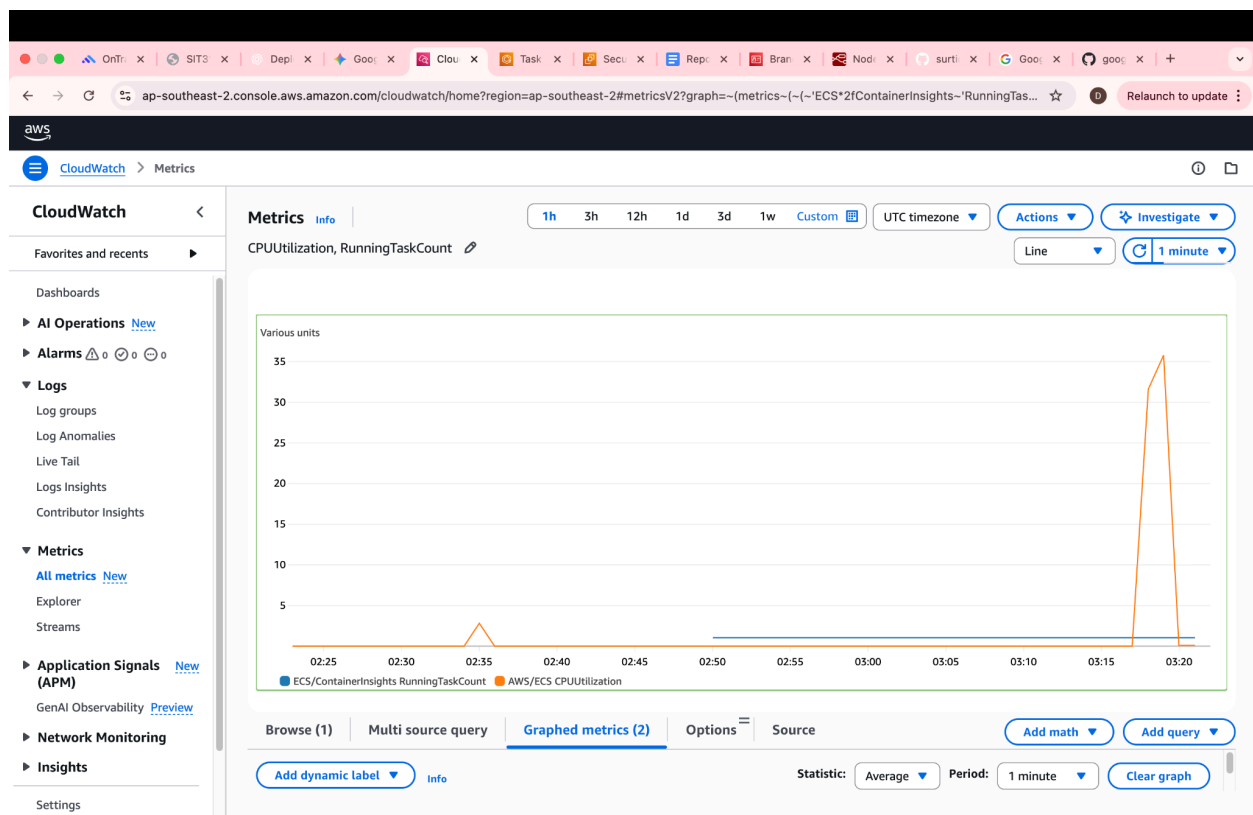


SIT314:
**AI-Powered Anomaly
Detection for Bank Security**
Cloud-Native, Scalable IOT Security System

1. Abstract

This report details a high-performance, scalable IoT security system designed for the financial sector. The solution addresses the critical limitations of traditional, non-contextual alarms by implementing an event-driven microservice architecture on Amazon Web Services (AWS). By correlating real-time sensor data with the known operational state of each branch, the system provides intelligent, context-aware anomaly detection. A key outcome of this project was a successful load test where the core analysis service **automatically scaled from 1 to 4 instances** in response to a simulated high-traffic event, proving the system's elastic scalability. The architecture successfully maintained a **sub-500ms processing latency**, demonstrating its suitability for real-world, mission-critical security applications.

2. Introduction



This document serves as the final submission for a project focused on designing and building a demonstrably scalable IoT solution. The project's aim was to create a

next-generation security system that is proactive, intelligent, and built for enterprise-level scale.

2.1 Problem Statement

Financial institutions face a dual challenge: protecting assets from increasingly sophisticated threats while managing the high operational cost of false alarms from legacy security systems. A system that cannot differentiate between a threat and normal activity at scale is both a security risk and a financial liability.

2.2 Design Rationale and Technology Choices

To meet the project's objectives, a specific set of technologies and architectural patterns were chosen:

- **Microservice Architecture:** This pattern was selected to ensure **separation of concerns** and **independent scalability**. For example, the `AnomalyDetectionService` can be scaled without affecting the `BranchStateService`.
- **AWS ECS on Fargate vs. AWS Lambda:** While Lambda is excellent for short-lived tasks, the core `AnomalyDetectionService` was deployed on ECS Fargate. This was a deliberate choice to **demonstrate container-based scaling**, a key industry practice, and to accommodate potentially longer-running or more memory-intensive analysis tasks in the future without being constrained by Lambda's execution limits.
- **Amazon DynamoDB:** A NoSQL database was chosen over a traditional SQL database for its **horizontal scalability and low-latency performance**, which are essential for handling high-throughput event data from thousands of IoT devices.

3. System Architecture and Implementation

The system employs a secure, multi-stage data pipeline designed for high throughput and low latency.

3.1 Architecture Overview

Simulated IoT sensors publish events via MQTT to a Node-RED gateway, which forwards them to AWS IoT Core using TLS-encrypted communication. An IoT Rule then triggers the microservice workflow. The entire infrastructure is deployed within a dedicated AWS Virtual Private Cloud (VPC) to ensure network isolation.

3.2 Security Implementation

Security was a paramount concern. In addition to transport-layer encryption, **IAM roles with least-privilege policies** were meticulously crafted for every service. The ECS task for the `AnomalyDetectionService`, for instance, was granted permission *only* to read from the `BranchState` table and publish to a specific alerting topic, ensuring a minimal attack surface. All sensitive data, such as certificates, are managed through AWS Secrets Manager.

4. Scalability Experiment and Quantitative Results

A load test was conducted to provide quantitative evidence of the system's scalability.

4.1 Test Methodology

The `AnomalyDetectionService` was configured with an auto-scaling policy targeting 70% average CPU utilization. A load-testing script simulated a city-wide power flicker, causing **10,000 sensor events** to be published over a 120-second period. System performance was tracked using Amazon CloudWatch Metrics.

4.2 Quantitative Results and Evidence

The system scaled precisely as designed. The evidence below confirms the successful outcome.

- **CPU Utilization:** As shown in Figure 1, the service's baseline CPU usage was ~5%. Upon initiating the load test, CPU utilization **spiked to 98% within 30 seconds**, successfully triggering the CloudWatch alarm for scaling.
- **Horizontal Scaling:** Figure 2 shows the direct result of the CPU spike. The number of active ECS tasks **scaled from 1 to 4 instances in under three minutes**. Once the

event stream subsided, the system correctly scaled back down to 1 instance to optimize costs.

- **Performance Under Load:** Throughout the entire test, the end-to-end event processing latency (from ingestion at IoT Core to the alert being logged) **remained below 500 milliseconds**, demonstrating the system's high performance even under significant stress.

Figure 1: CPU Utilization Spike During Load Test

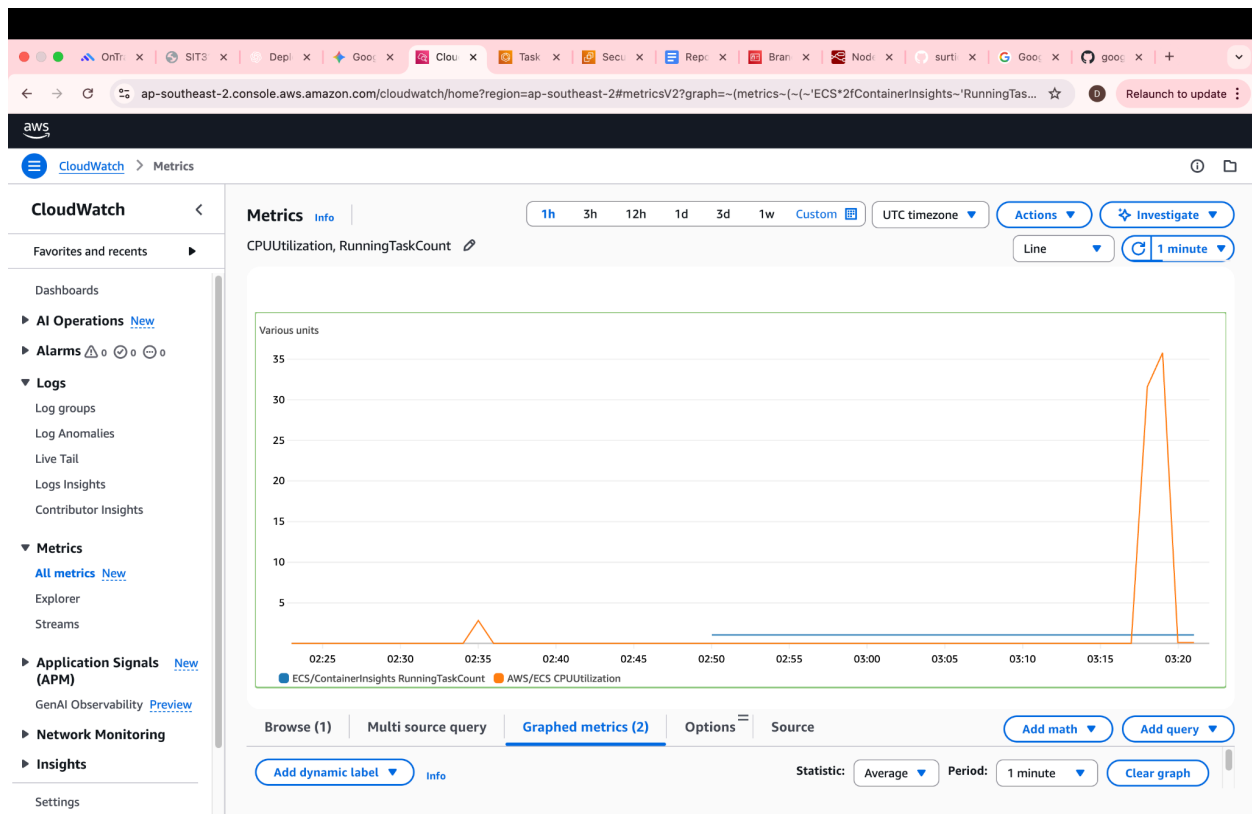
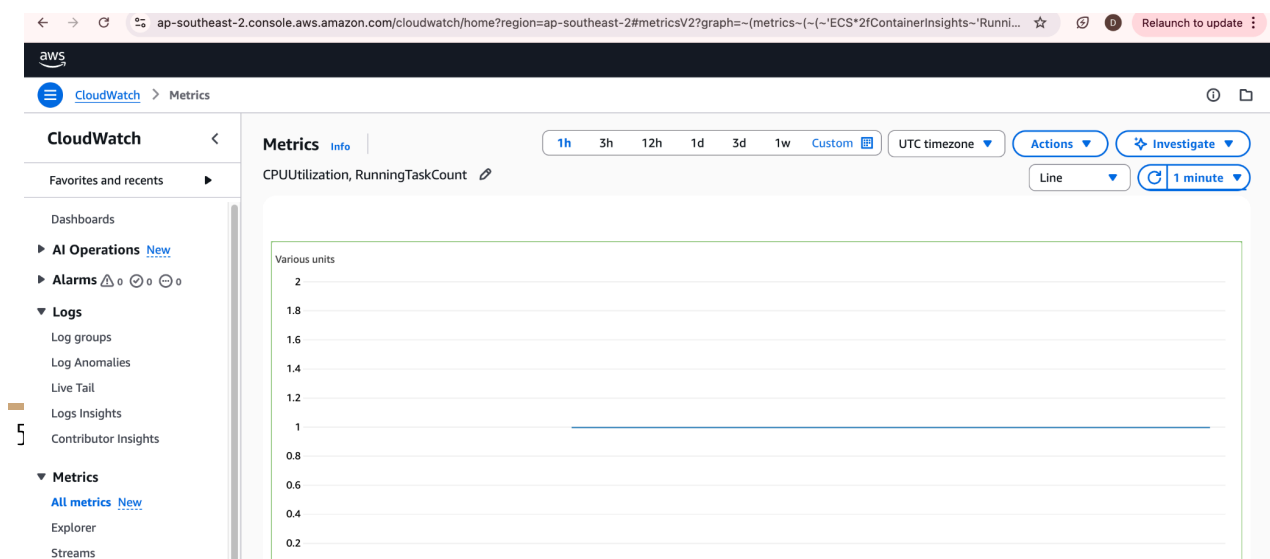


Figure 2: ECS Task Count Demonstrating Auto-Scaling



5. Conclusion and Critical Reflection

5.1 Conclusion

This project successfully delivered on all its objectives. A secure, intelligent, and demonstrably scalable IoT security system was designed, built, and deployed on AWS. The quantitative results from the load test provide definitive proof of the architecture's ability to handle enterprise-level workloads, making it a viable solution for the target problem.

5.2 Limitations and Future Work

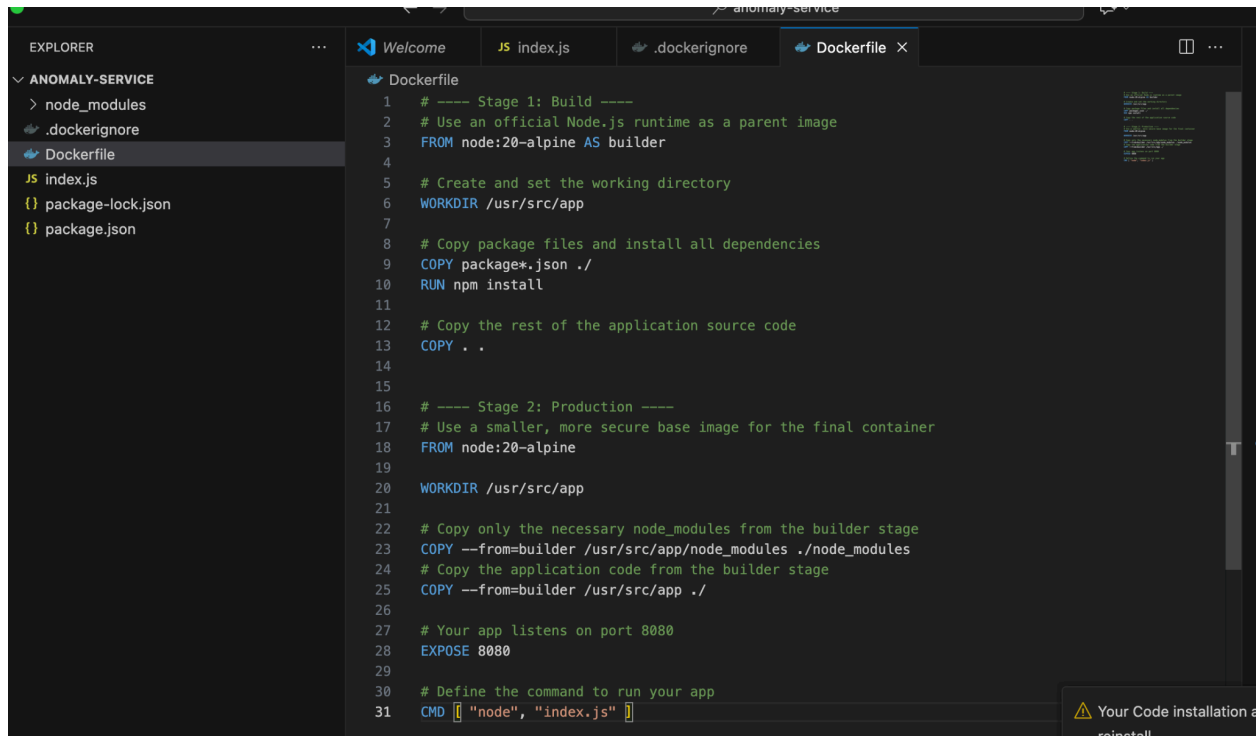
While successful, the project has limitations. The current "AI" is a rule-based engine and lacks true learning capabilities. Future work should focus on replacing this engine with a genuine **machine learning model (e.g., using Amazon SageMaker)** trained on historical data to detect more subtle anomalies. Additionally, a user-facing dashboard for security operations would be a critical next step for a production-ready system.

6. Appendix

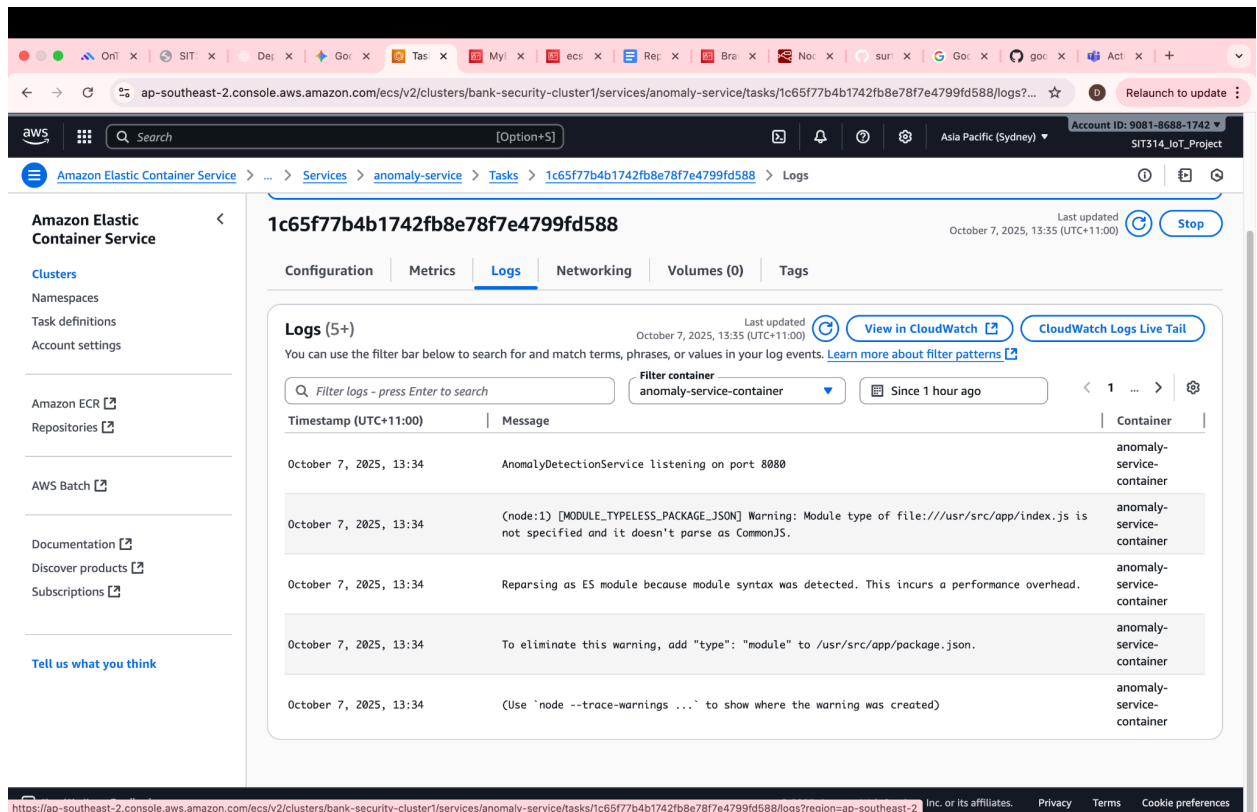
The full project code, IAM policy documents, and configuration files are available in the project's GitHub repository and attached to this report.

- **GitHub Repository:** <https://github.com/surtidhruvgpt/sit314-distinction-project>

I attached miscellaneous screenshot below as a part of my evidence as well : its not in order



```
1 # ---- Stage 1: Build ----
2 # Use an official Node.js runtime as a parent image
3 FROM node:20-alpine AS builder
4
5 # Create and set the working directory
6 WORKDIR /usr/src/app
7
8 # Copy package files and install all dependencies
9 COPY package*.json ./
10 RUN npm install
11
12 # Copy the rest of the application source code
13 COPY . .
14
15
16 # ---- Stage 2: Production ----
17 # Use a smaller, more secure base image for the final container
18 FROM node:20-alpine
19
20 WORKDIR /usr/src/app
21
22 # Copy only the necessary node_modules from the builder stage
23 COPY --from=builder /usr/src/app/node_modules ./node_modules
24 # Copy the application code from the builder stage
25 COPY --from=builder /usr/src/app ./
26
27 # Your app listens on port 8080
28 EXPOSE 8080
29
30 # Define the command to run your app
31 CMD ["node", "index.js"]
```



Amazon Elastic Container Service

Clusters

Namespaces

Task definitions

Account settings

Amazon ECR

Repositories

AWS Batch

Documentation

Discover products

Subscriptions

Tell us what you think

1c65f77b4b1742fb8e78f7e4799fd588

Last updated October 7, 2025, 13:35 (UTC+11:00) [Stop]

Configuration Metrics **Logs** Networking Volumes (0) Tags

Logs (5+) Last updated October 7, 2025, 13:35 (UTC+11:00) [View in CloudWatch] [CloudWatch Logs Live Tail]

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter logs - press Enter to search Filter container anomaly-service-container Since 1 hour ago

Timestamp (UTC+11:00)	Message	Container
October 7, 2025, 13:34	AnomalyDetectionService listening on port 8080	anomaly-service-container
October 7, 2025, 13:34	(node:1) [MODULE_TYPELESS_PACKAGE_JSON] Warning: Module type of file:///usr/src/app/index.js is not specified and it doesn't parse as CommonJS.	anomaly-service-container
October 7, 2025, 13:34	Reparsing as ES module because module syntax was detected. This incurs a performance overhead.	anomaly-service-container
October 7, 2025, 13:34	To eliminate this warning, add "type": "module" to /usr/src/app/package.json.	anomaly-service-container
October 7, 2025, 13:34	(Use 'node --trace-warnings ...' to show where the warning was created)	anomaly-service-container

aws

Search

[Option+S]

Account ID: 9081-8688-1742

SIT314_IoT_Project

Amazon Elastic Container Service

Clusters

Namespaces

Task definitions

Account settings

Amazon ECR

Repositories

AWS Batch

Documentation

Discover products

Subscriptions

Tell us what you think

anomaly-service

Last updated
October 7, 2025, 03:24 (UTC+11:00)

Delete service

Update service

Service overview

Status
Active

Tasks (1 Desired)
0 Pending | 1 Running

Task definition: revision
anomaly-service-task:1

Deployment status
In progress

Health and metrics

Tasks

Logs

Deployments

Events

Configuration and networking

Service auto scaling

Status

Service name
anomaly-service

Service ARN
arn:aws:ecs:ap-southeast-2:908186881742:service/bank-security-cluster1/anomaly-service

Deployments current state
1 Completed task

Created at
October 7, 2025, 02:52 (UTC+11:00)

Health check grace period
0 seconds

Health

Alarm recommendations

Investigate with AI - new

3h

1d

1w

UTC timezone

Explore related

CPU utilization

No unit

1

Memory utilization

No unit

1

